

CS151 Intro to Data Structures

Recursion
Binary Search

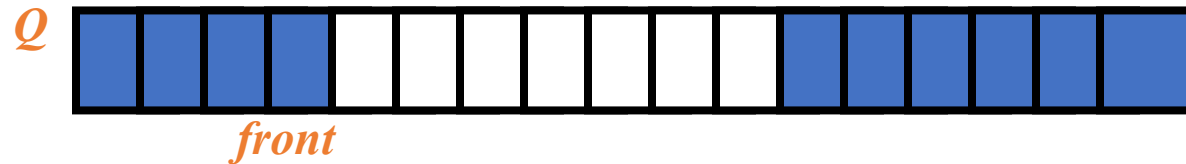
warmup: what is printed?

```
List<String> words =  
    new ArrayList<>(Arrays.asList("apple", "banana", "cherry", "date", "elderberry"));  
  
Iterator<String> it = words.iterator();  
while (it.hasNext()) {  
    String word = it.next();  
    if (word.length() % 2 == 0) {  
        it.remove();  
    }  
}  
  
System.out.println(words);
```

Circular Array Queue

- enqueued element inserted at $(\text{front} + \text{size}) \% n$
- dequeued element comes from front

wrapped-around configuration



Warmup 2:

A queue with $n = 5$ is in this state:

$a = [P, Q, R, S, T]$, $\text{front} = 3$, $\text{size} = 2$

- a. Which elements are actually in the queue, in order?
- b. What is the fewest number of operations that could have produced this state? Give a sequence.
- c. Which element was enqueued most recently?

Announcements

- Lab5 due today
- chance to resubmit assignments over fall break...
- Midterm Thursday after break (Oct 20th)

Outline

- Deques review
- Recursion Review
- Binary Search

Queues (FIFO)

- Restaurant Waitlist
 - FIFO make sense!
 - What if we pop off a party in the waitlist but then a table wasn't ready... we need to add them back in... push will put them at the end!
- Solution? **Deque**
 - gives us more flexibility

```
public interface Deque<E> {  
  
    //returns number of elements in the deque  
    int size();  
  
    //returns true if the deque is empty, false otherwise  
    boolean isEmpty();  
  
    //returns top element in the deque (or null if empty)  
    E first();  
  
    //returns top element in the deque (or null if empty)  
    E last();  
  
    //inserts the element e to the beginning  
    void addFirst(E e);  
  
    //adds the element e to the end  
    void addLast(E e);  
  
    //returns the first element in the deque (or null if empty)  
    E removeFirst();  
  
    //returns the last element in the deque (or null if empty)  
    E removeLast();  
}
```

Front and Back Markers

addFirst:

$f = (f - 1 + n) \% n;$

addLast:

$r = (r + 1) \% n;$

removeFirst:

$f = (f + 1) \% n;$

removeLast:

$r = (r - 1 + n) \% n;$

where n is size of the array

- f starts at 0, r starts at $n-1$
- Update f/r before updating the arr
- addFirst(A)
 - $f = (0-1)\%5 = 4$
 - $data[4] = A$
- addFirst(B)
 - $f = (4-1)\%5 = 3$
 - $data[3] = B$
- addLast(C)
 - $r = (4+1)\%5 = 0$
 - $data[0] = C$
- addLast(D)
 - $r = (0+1)\%5 = 1$
 - $data[1] = D$
- addLast(E)
 - $r = (1+1)\%5 = 2$
 - $data[2] = E$

Dequeues

In a deque with the previous update equations and f initialized to 0 and r initialized to $n-1$, what relationship between f and r indicate that:

- the dequeue is empty
 - f and r differ by 1
 - $(r+1)\%n == f$
 - Is this the only time they differ by one?
 - NO! When the dequeue is full!
- the dequeue is full
 - There is no relationship between f and r that can distinguish between empty and full
 - Option 1: $(r+2)\%n == f$ and leave an empty spot
 - Option 2: Keep a `NUM_ELEMS` variable

Outline

- Deques review
- **Recursion**
- Binary Search

Recursive functions – base case

Conditional statement that prevents infinite repetitions

Usually handles cases where:

- input is empty

- problem is at its smallest size

Recursion Example – Contains letter

Write a method called “containsLetter” that determines if a String contains a given character

Question: What are the parameters?

1. The character to look for
2. The string to be looking in

Question: What is the return type?

Code it!

Recursion Visualization – Contains letter

```
contains("l", "apple") =  
    contains("l", "apple")  
        contains("l", "pple")  
            contains("l", "ple")  
                contains("l", "le")  
                    return true
```

Recursive Method

Break problem down into smaller subproblem that we can repeat

Base case(s):

- no recursive calls are performed
- every chain of recursive calls must reach a base case eventually

Recursive calls:

- Calls to the same method in a way that progress is made towards a base case
- Often called “the rule”

Outline

- deques review
- Recursion
- **Binary Search**

Binary Search

- efficient search in a sorted list
- can be implemented **recursively**

Search steps:

1. Calculate midpoint
2. Compare the value at the midpoint with the target value
 - a. if equal:
 - i. return index
 - b. if target value $<$ midpoint value:
 - i. **search** the left portion of the list
 - c. if target value $>$ midpoint value:
 - i. **search** the right portion of the list

Binary Search

Search for an integer (22) in an ordered list

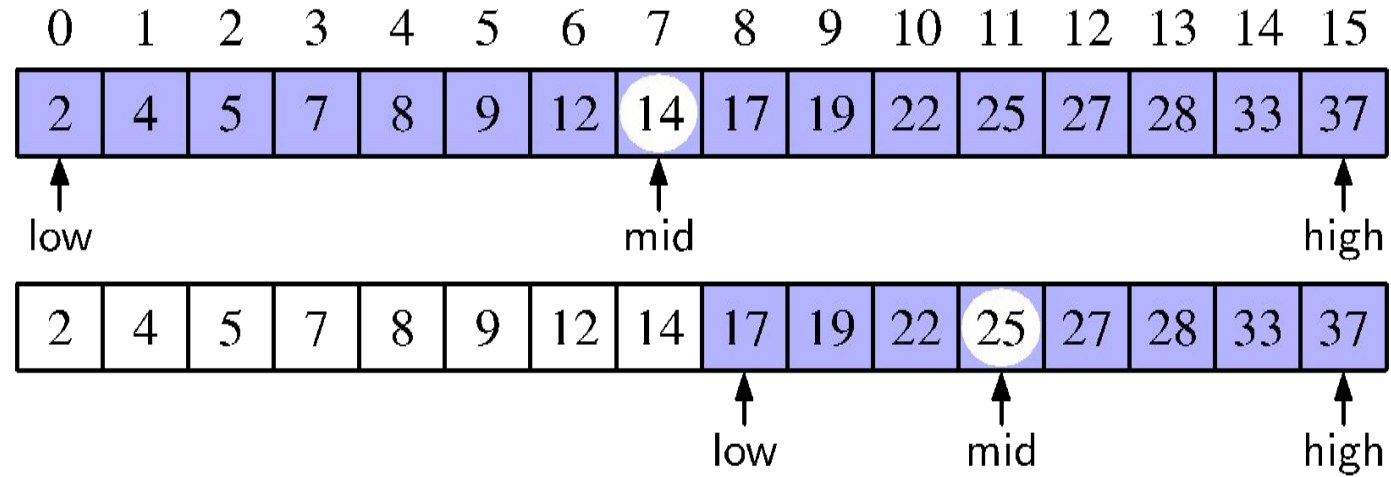
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2	4	5	7	8	9	12	14	17	19	22	25	27	28	33	37

target = 22

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
2	4	5	7	8	9	12	14	17	19	22	25	27	28	33	37
↑							↑								↑
low							mid								high

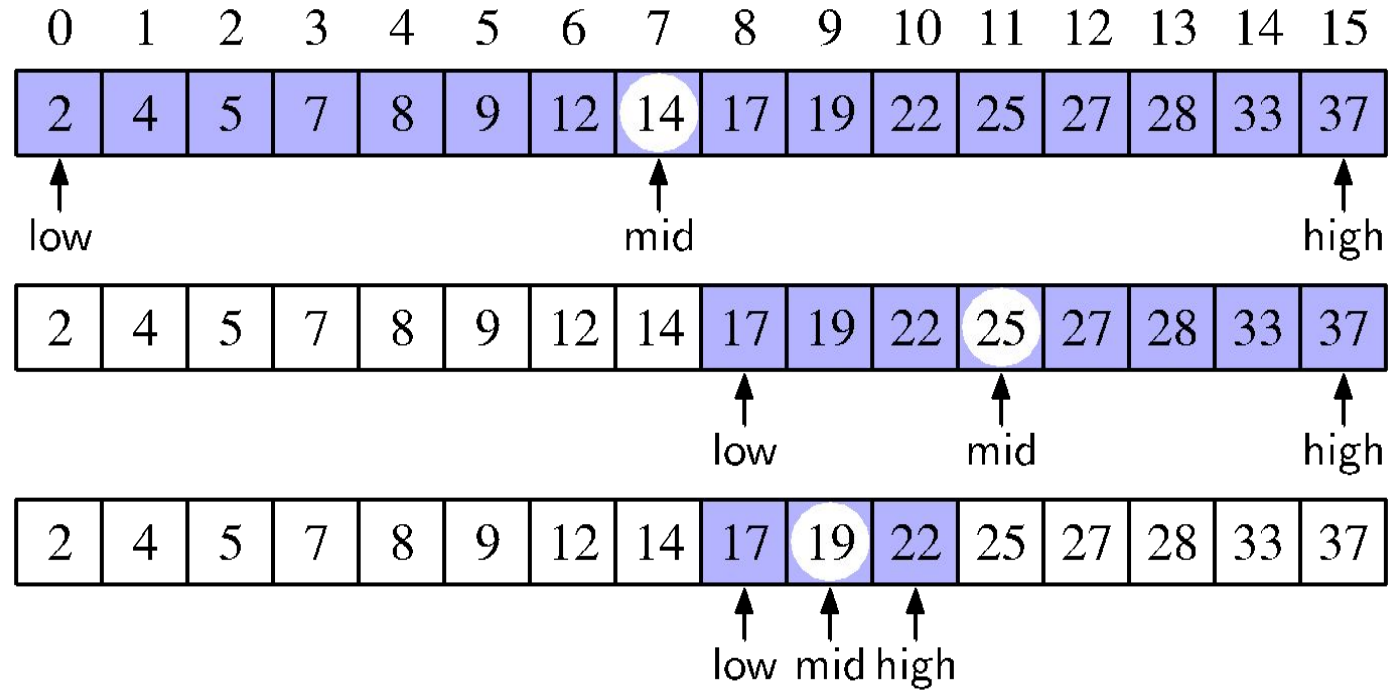
```
int mid = (high + low) / 2; //index  
int midVal = arr[mid]; //14
```

target = 22

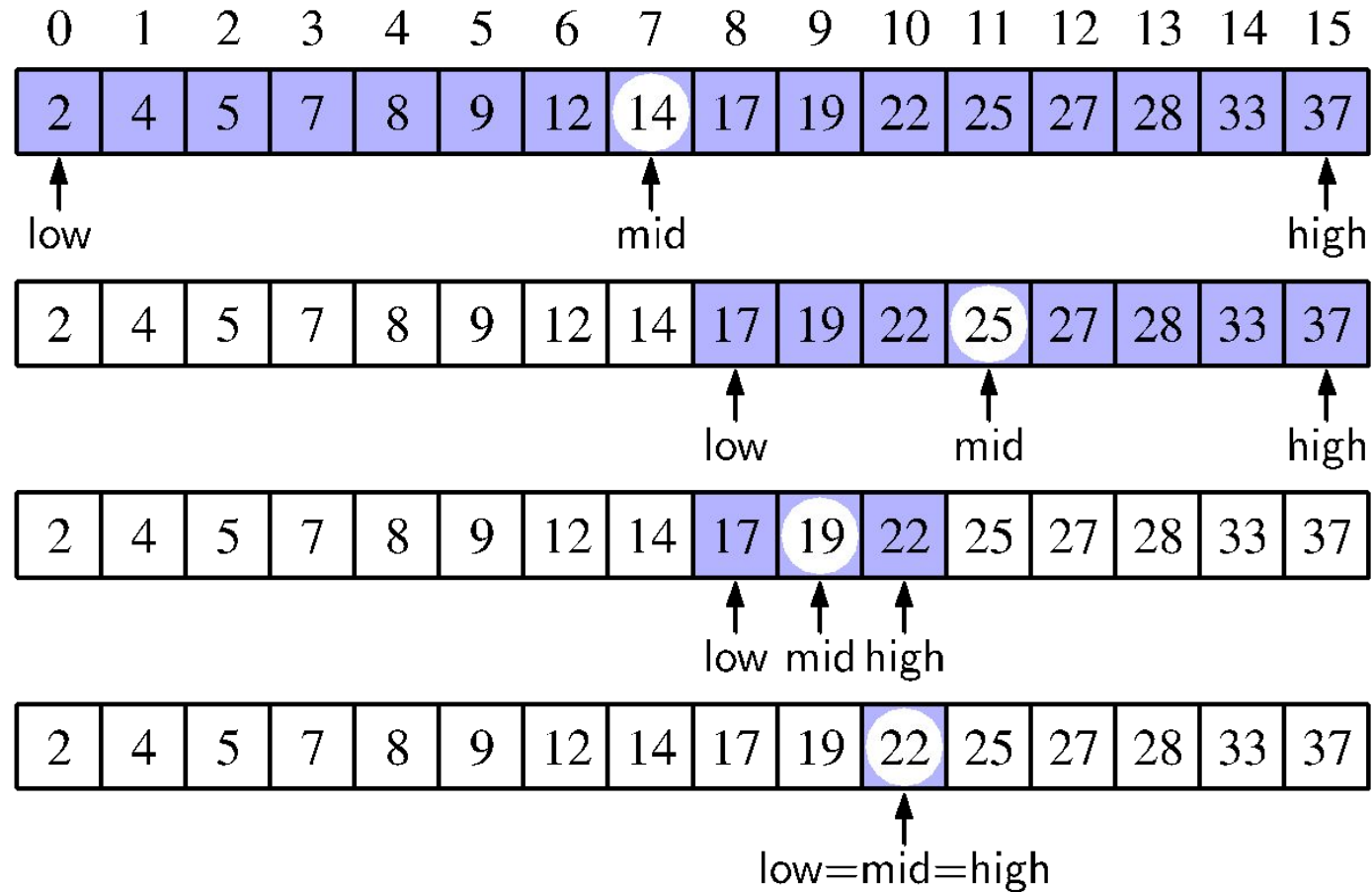


low = mid+1;

target = 22



target = 22



Binary Search Implementation

Binary Search Analysis

Each recursive call divides the array in half

If the array is of size n , it divides (and searches) at most $\log n$ times before the current half is of size 1

$O(\log n)$

Comparable

Binary search on a list of objects requires that the objects have natural ordering

In other words, the objects must implement `Comparable`

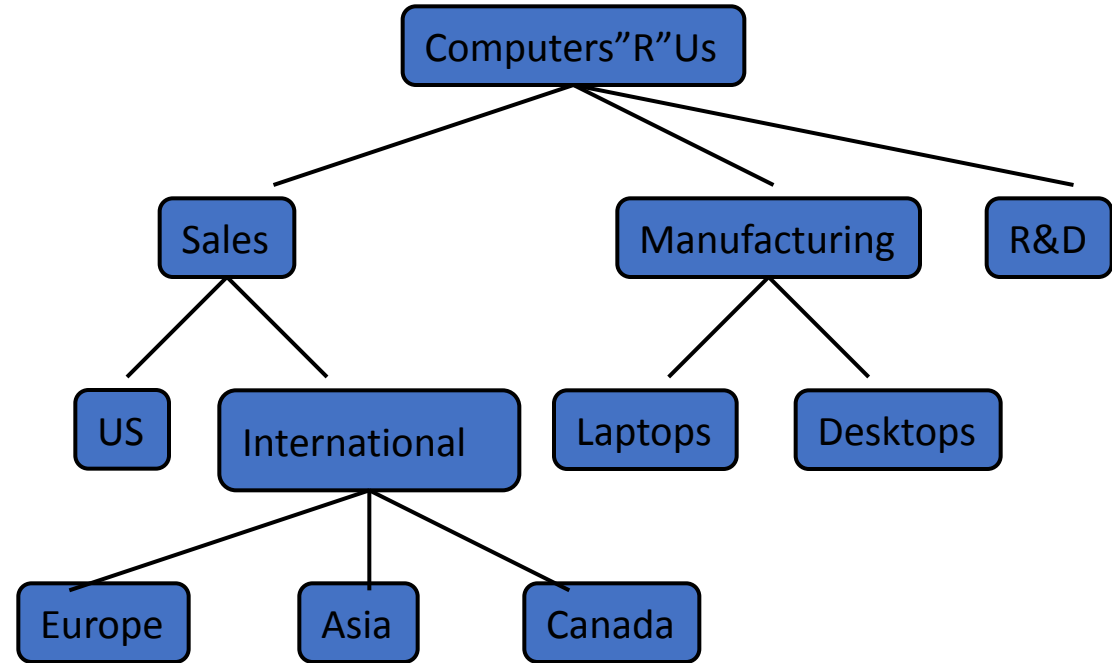
`compareTo`

Tree

A tree is a **hierarchical** data structure

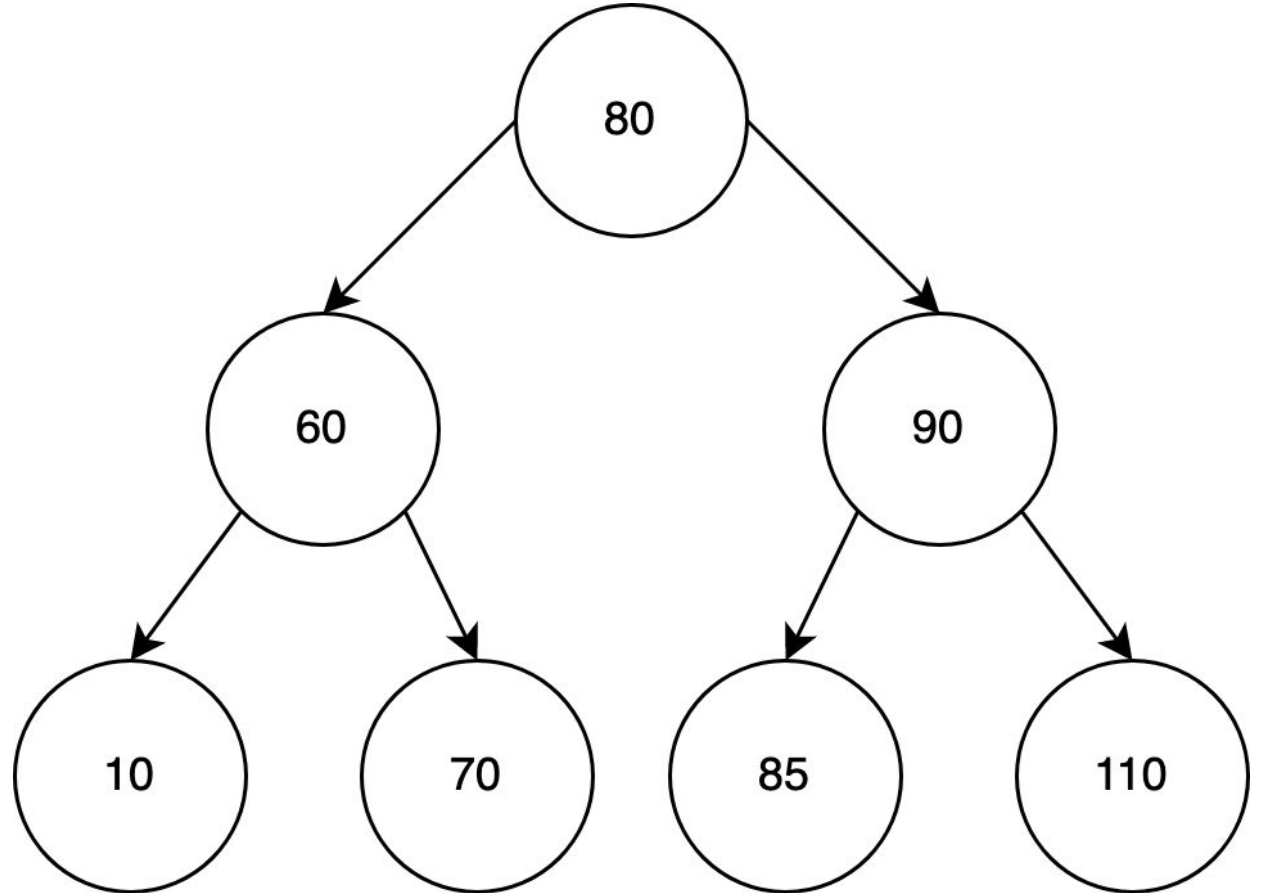
Node: individual elements in the tree

Nodes have a parent-child relation



Trees: Nodes

```
class Node {  
    int key;  
    Node left;  
    Node right;  
  
    public Node(int item) {  
        key = item;  
        left = null;  
        right = null;  
    }  
}
```



Terminology

root: no parent

A

external/leaf node: no children

E, I, J, K, G, H, D

internal node: - node with at least one child

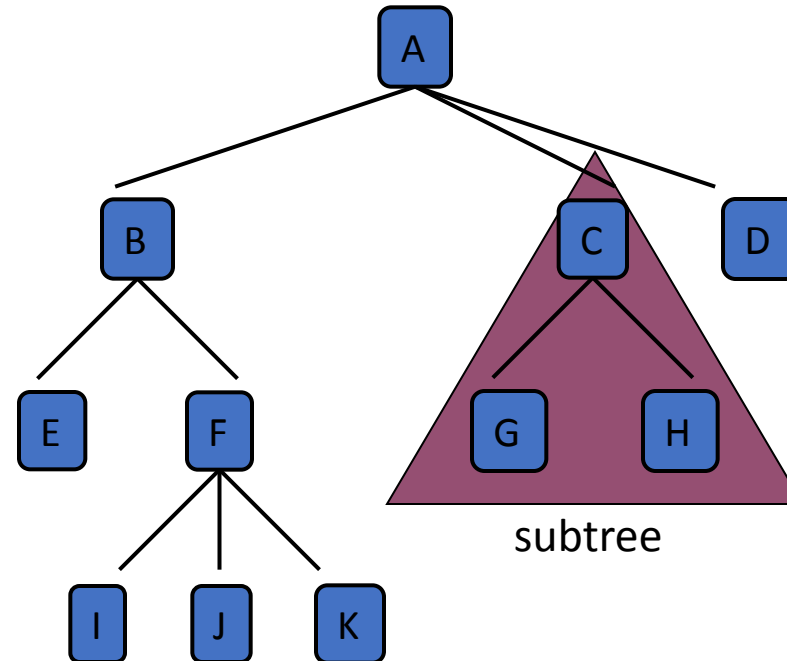
A, B, C, F

parent/child

depth - # of ancestors

Height - Maximum number of edges from a leaf node to the root

- **Subtree:** tree consisting of a node and its descendants



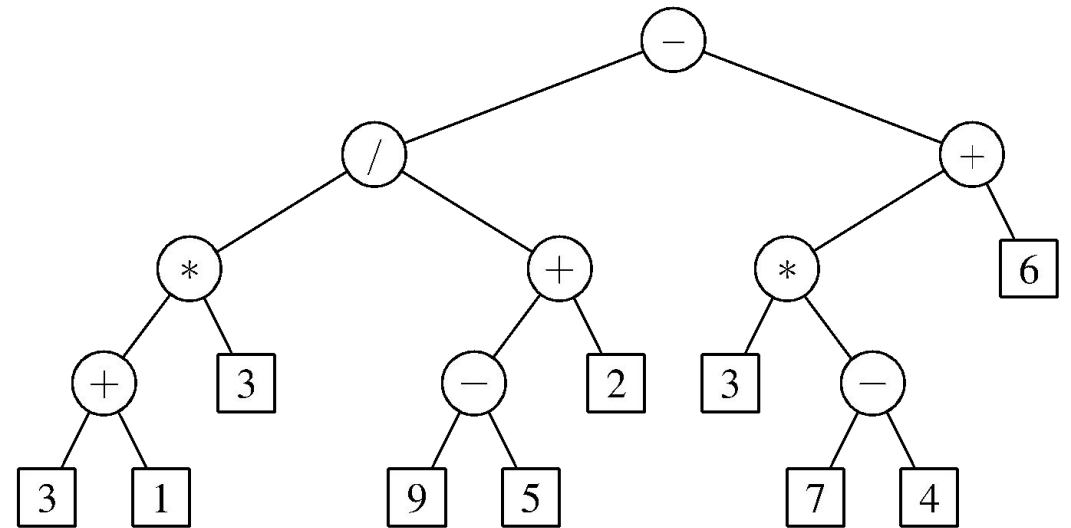
Binary Tree

Each node in a **binary tree** has at most two children

Recursive definition:

Each node has at most two children

- Both subtrees are **binary trees**

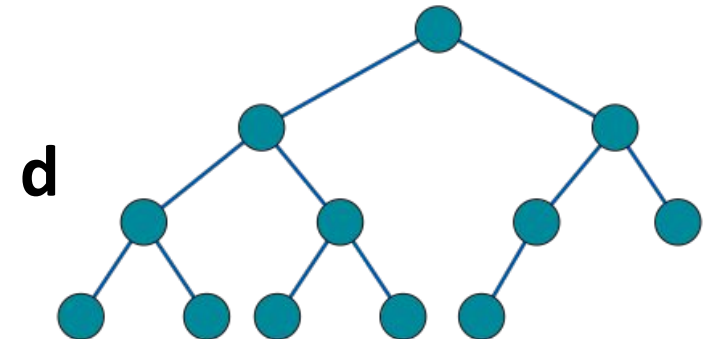
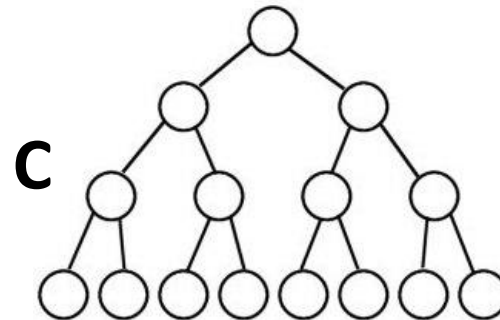
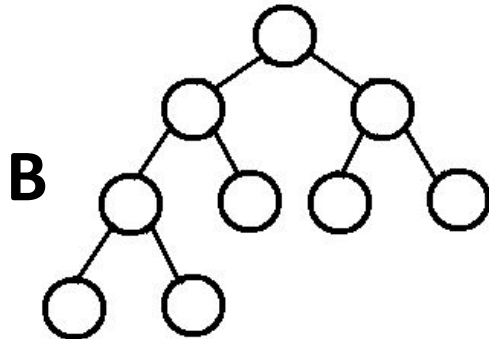
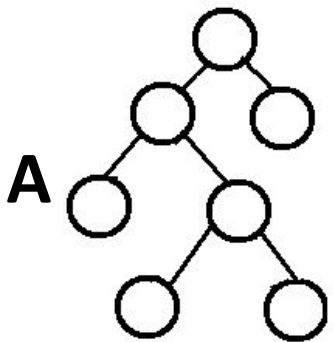


Types of Binary Trees

A binary tree is **full** (or proper) if each node has **zero or two children**

A binary tree is **complete** if every level (except possibly the last) is fully filled and, if the last level of the tree is not fully filled, the nodes of that level are filled from left to right

If a complete binary tree is filled at every level, it is **perfect**



Types of Binary Trees

A binary tree is **full** (or proper) if each node has **zero or two children**

A binary tree is **complete** if every level (except possibly the last) is fully filled and leftmost aligned

If a complete binary tree is filled at every level, it is **perfect**

Q1: Is every full binary tree a complete binary tree?

Q2: Is every complete binary tree a full binary tree?

Q3: Is every perfect binary tree a full binary tree?

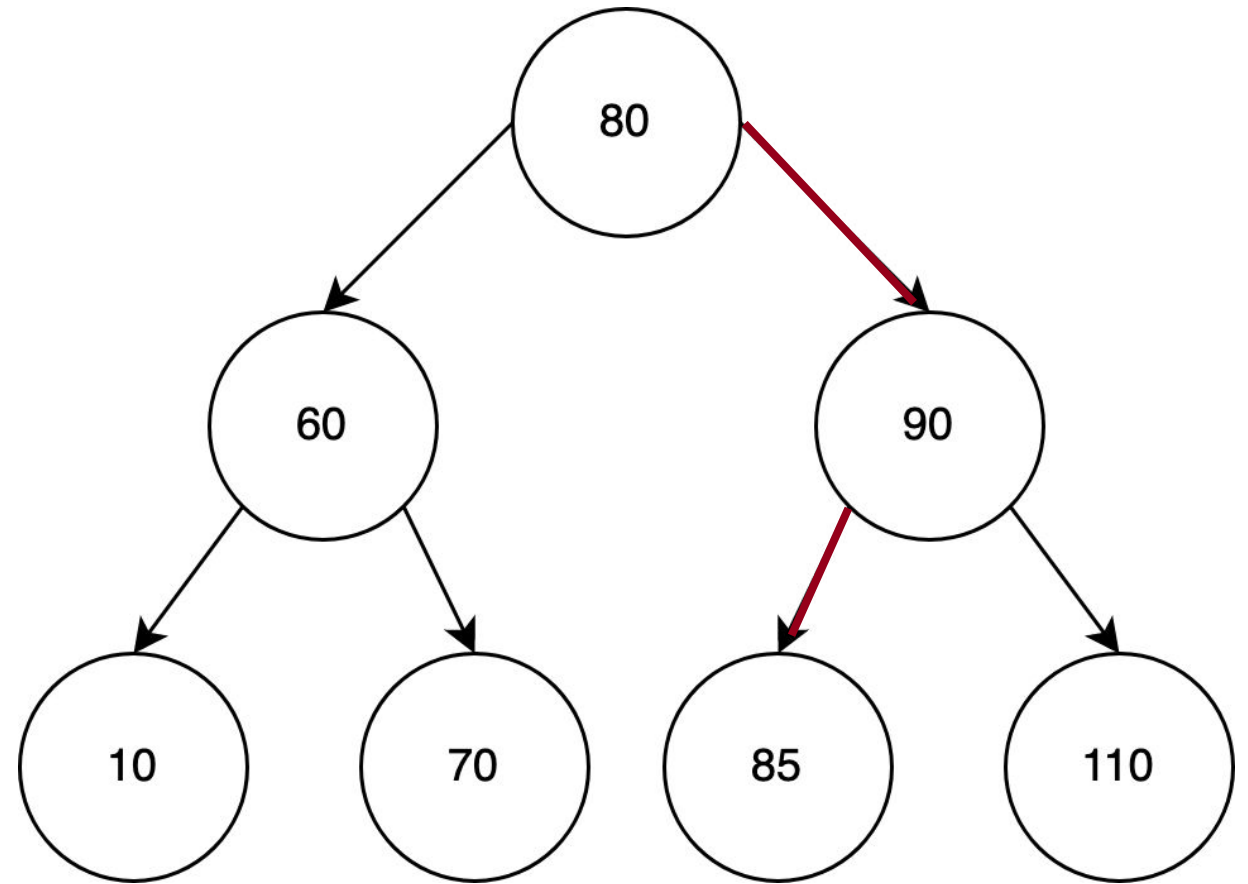
Binary Trees: Height

Height of a tree:

Maximum number of edges from a leaf node to the root

Height? 2

$$\log_2(7) \approx 2$$

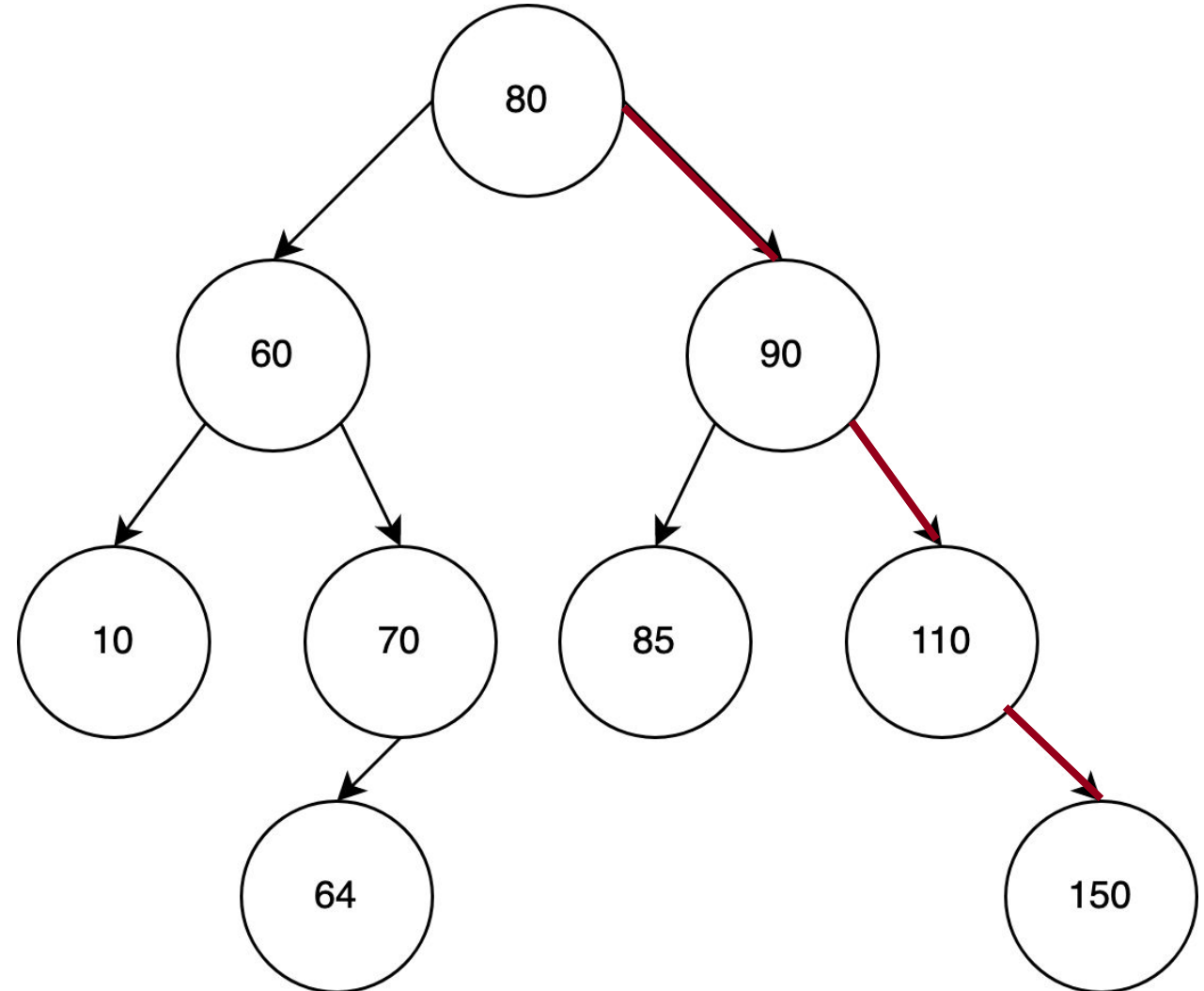


Tree Review

Height? 3

$$\log_2(9) \approx 3$$

Height of a binary tree is roughly $\log(n)$ where n is number of nodes

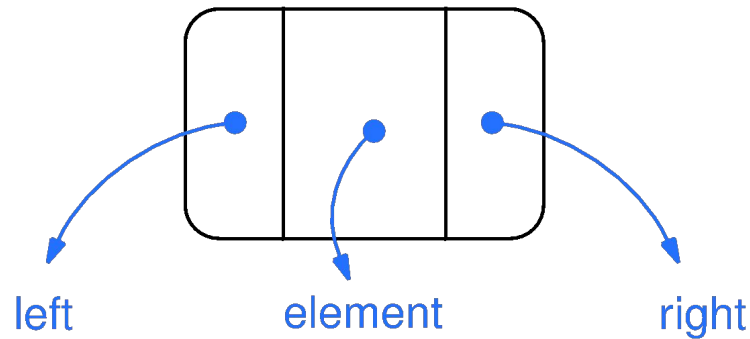


Binary Tree Interface

```
public interface BinaryTree<E> extends
Comparable<E>> {
    int size();
    boolean isEmpty();
    void insert(E element);
    boolean contains(E element);
    ...
}
```

Node Implementation

```
public class Node<E> {  
    private E element;  
    private Node<E> left;  
    private Node<E> right;  
    //constructors, getters, setters  
}  
}
```



Class

```
public class LinkedBinaryTree<E> extends  
Comparable<E>> implements BinaryTree<E> {  
    // what instance variables?  
    // nested Node class  
  
}
```

Summary

- iterators
 - What is the **Iterable** interface? `iterator()`
 - What is the **Iterator** interface?
 - `next()`, `hasNext()`
- Binary Search
 - runtime complexity? $O(\log n)$
 - more, less, or equal efficiency to a linear search?
 - $O(n)$
 - $O(\log n)$